

ISLAMIC UNIVERSITY OF TECHNOLOGY

EEE 4602 Project

Course Name: Signals and Systems Lab

Members:

- 210021139: Mashrur Saad Emad
- 210021143: Bireer Ibrahim Elhag
- 210021151: Md. Shazid Hasan
- 210021163: Radiba Nuzhat

Section: A1

Date of Submission: 23-Jul-25

Introduction

This project is about developing an interactive Graphical User Interface (GUI) system designed to help users intuitively understand the convolution operation . As we know, Convolution is a fundamental concept in signal processing. It's used in applications like filtering, system analysis & image processing. However understanding how convolution works can be challenging due to its mathematical nature. This GUI tool provides an intuitive visualization of the convolution process which helps to understand the concept through real-time animation.The key Features we have added on to create the projects are firstly Signal Mode Selection.Here we use (a) Discrete-time signals: users can input signal vectors and specify their starting indices (b) Continuous-time signals: users can choose from standard signal types and define their parameters (amplitude, start, end time).Then we developed the Interactive Visualization part.Then the User Controls shows the Compute Convolution, resetting the values and controlling the speed.Finally we added the bonus feature of Correlation Computation. All these steps will be discussed properly in the further report.

Brief Summary of the Problem Statement

The project requires to design & implement a graphical user interface (GUI) that enables users to visualie convolution operation between 2 signals an input signal & an impulse response. The application must support 2 modes, discrete-time and continuous-time modes. In discrete mode, users manually input signal vectors and their start indices. In continuous mode, users select signal shapes like impulse, step, triangular, rectangular & sawtooth, and specify parameters like start time, end time & amplitude. And the compute convolution control for starting animation.

The GUI should allow users to animate the convolution process & display 3 synchronized plots: (i) the fixed input signal, (ii) the mirrored and moving real time impulse response, (iii) the animated real-time output signal. The application must also include input validation. And for extra bonus mark, animation speed control & a reset feature can be used & correlation computation option should be available as well.

Problem Review:

The knowledge required for this project is -

1. **Theoretical**: Convolution of -
    a. Discrete time signal: Formula: $y[n] = \sum_{k=-\infty}^{\infty} x[k] \cdot h[n - kx]$ and need to know about the visualization as well.
    b. Continuous time signal: Formula: $y(t) = \int_{-\infty}^{\infty} x(\tau) \cdot h(t - \tau)\, d$ and need to know about the visualization as well.
2. **Software**: Python -
    a. Coding: functions, loops, NumPy arrays, and logic for signal manipulation.
    b. Operation of GUI: how to design and manage interactive GUI components using Tkinter and embed visual plots using Matplotlib.

Methodology:

At first the libraries were imported. The used libraries were - numpy, tkinter, matplotlib, threading etc. Then class innit function defined as per syntax. Then the main coding part was done -

1. **Interface**: At first the interface of the GUI was designed. Here the dark background was used and the position of controls and graphs etc all these were designed. Here buttons, labels, entry, frames, texts etc were designed.
   Functions used:
   - setup_gui
2. **Inputs**: The inputs, buttons and controls were designed. Which button to be pushed for controlling what and how the inputs should be taken and as well as validating the inputs as well. Also the continuous signal types were defined here as well - impulse, step, rectangular, triangular and sawtooth.
   Functions used:
   - on_mode_change
   - setup_discrete_inputs
   - setup_continuous_inputs
   - validate_discrete_inputs
   - validate_continuous_inputs
   - generate_continuous_signal
3. **Computation**: Here using the built in function or formula, the computation of convolution was done. In case of discrete convolution, built in function was used. But we faced some errors while trying to compute the continuous function using the built in convolve function. So we used the theoretical continuous convolution formula.
   Functions used:
   - compute_convolution
   - compute_discrete_convolution
   - compute_continuous_convolution
4. **Animation**: Here the animation part was designed. There were three plots, one was the static input, second one the moving impulse response and the third one the resultant convoluted signal. At first the shift & overlap variables were created and using those the moving impulse response was found. And then using the computed continuous convolution earlier, the result was also found. Threading was also used to allow animations to run smoothly in the background without freezing the GUI interface.
   Functions used:
   - animate_discrete_convolution
   - animate_continuous_convolution
5. **Extra for Bonus (Correlation & Reset)**: Here the correlations were computed and animated in a similar manner and reset function was done to stop any running animation, clear all plots, and then reset the GUI inputs and settings to their default state by calling on_mode_change().
   Functions used:
   - compute_correlation
   - animate_discrete_correlation
   - animate_continuous_correlation
   - reset_all

Design:

The GUI was designed using tkinter and matplotlib libraries. Designed in a way that the controls were in the left side and the plots were in the right side of the screen.

Initially the background was white and all the texts were written in a similar manner. Later the background was changed to black and the texts were also modified with large and bold variations.

In the controls there were buttons, radio buttons, panels, text and numerical input options, drop down option etc.

The radio button allows switching between discrete and continuous modes easily. There were input fields for signal types, amplitudes and time ranges. For continuous signal, dropdown menu was used to select the signal type and for discrete numerical input option was used to have an array. And there were bttons for computing convolution, correlation and reset functions.

For animation speed control a slider was also used to control the speed for interactive animation.

The visualization part on the right side had three parts -

1. The input signal
2. The moving impulse response
3. Real time convolution result.

Threading was used to keep the GUI responsive during animations by running them in a separate thread.

Warning messages were shown as well using messagebox to prevent overlap of operations.

Tool Used:

Tool used for the project was Python. None of the team members were much acquainted with python earlier. We initially decided to do the project on Matlab, but we were finding some difficulties to work with mlapp so we switched to python instead. So it was a new experience.

The skill development was on using different python libraries and functions. We used Tkinter, NumPy, Matplotlib.

We learned how to design a full-featured Graphical User Interface (GUI) using buttons, sliders, input fields, and plot canvases using tkinter.

Using matplotlib improved skills in plotting and animating signals, customizing axes, legends. And using stem, plot, and fill_between functions for clear visualization.

Using numpy allowed to work with the mathematical part and strengthened understanding of vectorized signal operations like convolve, correlate, arange and array indexing.

The problem solving skill was also developed a lot. Like in case of continuous convolution, while using the convolve function, the time axis was not matching in the resultant plot due to some synchronization error. So the code was modified using the theoretical continuous convolution formula. Thus there were many such sections where problem solving skills were improved.

Teamwork:

Student ID	Name	Contribution Area
210021139	Mashrur Saad Emad	• Research & Analysis • Interface Design • Computation • Animation • Report making
210021163	Radiba Nuzhat	• Research & Analysis • Input Handling • Computation • Correlation & Reset
210021151	Md. Shazid Hasan	• Interface Design • Input Handling • Correlation & Reset • Report making
210021143	Bireer Ibrahim Elhag	• Research & Analysis • Correlation & Reset

Timeline:

- May 30: Project announcement and group formation
- July 13: Problem analysis and planning
- July 14: Interface design
- July 15: Input handling and Computation coding part
- July 16: Animation and extra parts for bonus
- July 17: Debugging and minor changes
- July 18: Project presentation

Attachments:

https://drive.google.com/drive/folders/1f823r4ClODuClkL_tzBmBtJPz5xjgeut?usp=drive_link

Complete source code:
https://drive.google.com/drive/folders/1vFp6WTI1PV4OqPzvEu7yJEXszYagkLOk?usp=drive_link

Video File Showing how your project is working:
https://drive.google.com/drive/folders/1xh8BIOWlWqmlgC_QcIHsSy9Q9h3FkZFU?usp=drive_link